```
import heapq
# Node class
class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None
    # Compare nodes based on frequency
    def __lt__(self, other):
        return self.freq < other.freq
# Generate Huffman codes
def generate_codes(root, code, huffman_codes):
    if root is None:
        return
    if root.char is not None:
        huffman_codes[root.char] = code if code else "0"
        return
        generate_codes(root.left, code + "0", huffman_codes)
        generate_codes(root.right, code + "1", huffman_codes)
# Main program
text = input("Enter a string: ")
if not text:
    print("Please enter a non-empty string.")
else:
    # Calculate character frequencies
    frequency = {}
    for char in text:
        frequency[char] = frequency.get(char, 0) + 1
    # Create priority queue
    heap = []
```

```
    for char, freq in frequency.items():
        heapq.heappush(heap, Node(char, freq))
    # Build Huffman tree
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        new_node = Node(None, left.freq + right.freq)
        new_node.left = left
        new_node.right = right
        heapq.heappush(heap, new_node)
# Generate Huffman codes
    root = heap[0]
    huffman_codes = {}
    generate_codes(root, "", huffman_codes)
    # Display results
    print("\nCharacter Frequencies:")
    for char, freq in frequency.items():
        print(repr(char), ":", freq)
    print("\nHuffman Codes:")
    for char, code in huffman_codes.items():
        print(repr(char), ":", code)
   # Encode the input string
    encoded = ""
    for char in text:
        encoded += huffman_codes[char]
    print("\nEncoded String:", encoded)
```

Huffman Encoding is a lossless data compression technique that reduces the size of data by assigning variable-length binary codes to characters based on their frequency.

Characters that occur more frequently get shorter binary codes,Characters that occur less frequently get longer binary codes,No information is lost during compression.

A Greedy Strategy is an algorithmic technique that makes the best possible choice at each step, hoping to obtain an optimal overall solution.

In simple words:It selects the smallest or best available option at every step without reconsidering previous decisions. In Huffman Encoding:

1. Select the two characters with the lowest frequencies.
2. Combine their frequencies.
3. Create a new node.
4. Insert the new node back into the priority queue.
5. Repeat until only one node remains.

This is why Huffman Encoding uses a greedy approach. Let:

- \(n\) = total number of characters in the input.
- \(k\) = number of distinct characters.

Operation	**Time complexity**
Calculate frequency	O(n)
Build priority queue	O(k log k)
Build Huffman tree	O(k log k)
Generate Huffman codes	O(k)
Encode the string	O(n)

Overall time complexity**O(n + k log k)**

Auxiliary space complexity**O(k)**

The encoded output itself requires additional space proportional to its length.

Neither is universally better. It depends on the problem. Greedy is suitable when locally optimal choices lead to a global optimum. Dynamic Programming is useful when problems have overlapping subproblems and optimal substructure. heapq is a built-in Python module that implements a heap-based priority queue.

In our program, we use a Min-Heap.

A Min-Heap is a data structure in which the smallest element is always available at the top.In Huffman Encoding, we repeatedly need to select the two characters with the lowest frequencies,Instead of searching manually every time, we use a min-heap.

each node represents a character or a combination of characters in the Huffman tree. ef is used to define a function. __init__ is a special method called a constructor.It automatically runs when we create a new Node object.self refers to the current object. Huffman Encoding uses a binary tree. Every node can have a left child and a right child.

__lt__()?

It is a special Python method called the less-than method.It compares two Node objects based on their frequencies.

Let's trace the algorithm using the sample input text: **"BABA"**

Step 1: Calculate Character Frequencies

The code loops through the characters to build the frequency map:

- 'B' appears **2** times.
- 'A' appears **2** times.

Step 2 & 3: Priority Queue (Min-Heap)

The code inserts the nodes into the heap. Because it's a min-heap, it orders them dynamically based on frequency (using the custom __lt__ method we defined):

- heap = [Node('B', freq=2), Node('A', freq=2)]

Step 4: Build Huffman Tree

The code enters the while len(heap) > 1: loop.

- **Pop lowest:** left = Node('B', freq=2)
- **Pop next lowest:** right = Node('A', freq=2)
- **Create parent:** new_node = Node(None, freq = 2 + 2 = 4)
    - new_node.left points to 'B'
    - new_node.right points to 'A'
- **Push back:** The heap now contains just one parent root node (len(heap) == 1). The loop finishes.

Step 5: Code Generation (The Dynamic Tree Search)

The generate_codes(root, "", huffman_codes) function runs recursively.

- **Call 1:** Starts at root (freq=4, char=None) with path string "". It branches **Left**: calls generate_codes(Node('B'), "0", huffman_codes)
- **Call 2 (Left Branch):** Reaches Node('B'). Since root.char is not None, it records the character. **Result Saved:** huffman_codes['B'] = "0",It runs return to drop out of this function instance.
- **Call 1 resumes:** It now branches **Right**: calls generate_codes(Node('A'), "1", huffman_codes)
- **Call 3 (Right Branch):** Reaches Node('A'). Since root.char is not None, it records the character. **Result Saved:** huffman_codes['A'] = "1", It runs return.

Step 6: Encode the Input String

Your script runs this encoder loop: encoded = "".join(huffman_codes[char] for char in text)

- For text "BABA", it translates step-by-step to: "0" + "1" + "0" + "1" → **"0101"**